Similarity Self-Check Report

Agentic Retrieval-Augmented Generation over Live Workspace Data: A Survey and the Cortex Assistant

Self-check, not an official certificate. The project team produced this report with open tools and published sources. It is not a Turnitin, DrillBit/ShodhShuddhi, iThenticate or Urkund report, and it did not compare the text against any repository of student papers. Use it to find passages to revise before the official institutional check.

Similarity index (external sources)

0.2%

9 of 4603 checked words lie inside 8-word sequences found in a cited source or on the web

Excluding titles of cited works and venue names: **0.2%**

Document	docs/paper/paper.pdf (IEEE review paper, 7 pages)
Authors	Ajit Paraskar, Prathamesh Kumbhar, Harshvardhan Patil, Bikram Ghosh; guide Prof. Swapnali Gaikwad (Dr. D. Y. Patil Institute of Technology, Pimpri)
Checked on	31 October 2026
Words checked	4603 (after exclusions below)
Match unit	identical sequence of 8 or more words (case and punctuation ignored)
Self-overlap	0.9% of words also appear in the team's own report (same authors and project). Reported separately and **not** counted in the index.

48
unique cited references: 39 compared as arXiv full text, 0 by abstract, 9 by title only

1
sources with at least one 8-word match

43
exact-phrase web searches run for this document

0
web searches that found the phrase elsewhere

Top sources

#	Source	Text compared	Words	Share
1	GRASP: GRanularity-Aware Search Policy for Agentic RAG https://arxiv.org/abs/2607.10463	arXiv full text	9	0.20%

Share = document words inside matched sequences from that source ÷ words checked. A word matched by two sources counts once in the index.

What was checked and what was left out

- Excluded: Title and author/affiliation block
- Excluded: Bibliography (REFERENCES list, entries [1]-[n])
- Excluded: Monospace (code) spans
- **Source check (whole text):** every cited reference in refs.bib / biblo.bib was looked up in the Semantic Scholar Graph API; where an arXiv id existed, the full paper text was downloaded from arXiv. Every 8-word sequence of the document was compared with every source.
- **Web check (sample):** every sentence of 10 or more words (186 sentences); every 4th of them (47) was also searched on the web. For each, a distinctive 10-word span was searched as an exact phrase with Claude's WebSearch tool.
- **Cross-document check:** 8-word overlap with the team's own report.

Matched passages (external sources)

#	Page	Section	Words	Matched text	Assessment
1	2	III. A Taxonomy Of Agentic Rag	9	*"agentic rag as a finite horizon markov decision process"*	**wording shared with source: rephrase or quote**

Full text with matches highlighted

Coloured text with a number = matched an external source (number = row in Top sources). Dashed underline = also appears in the team's own report (self-overlap, not counted).

PAGE 1 · ABSTRACT

Retrieval-augmented generation (RAG) grounds a large language model (LLM) in retrieved text, but a single retrieve-then-read pass struggles with multi-hop questions, noisy passages and changing data. In agentic RAG the model controls retrieval itself. It can split a question, search more than once, pick a tool, judge its evidence and keep memory. We review this work around GRASP, which uses reinforcement learning to train a 3Bparameter policy that chooses, at each step, between semantic search, keyword search and reading a whole paragraph over a sentence-level index. We sort the literature into eight design dimensions and compare representative methods along them. As a case study we describe Cortex, an open-source assistant we built over Gmail, Google Calendar, Google Drive, Notion, Slack and GitHub. It plans sub-questions in one LLM call, searches in parallel, checks each cited claim against its evidence, keeps memories with provenance and expiry, and runs actions only after approval. On 12 questions over a synthetic workspace, the cited evidence held 92% of the expected key facts. Cortex declined to guess when no evidence existed, and no action ran without consent.

Index Terms—Retrieval-augmented generation, agentic RAG, large language models, search policy, retrieval granularity, reinforcement learning, citation verification, agent memory, tool use, Model Context Protocol

I. INTRODUCTION

An LLM's knowledge is frozen at training time and goes stale. RAG conditions the answer on passages fetched from an external corpus [1], usually in one pass, with a retriever ranking chunks and a generator reading the top few [2]. That works for one fact in a fixed collection. It fails when the second half of a question can only be searched after the first is answered [3], [4], when retrieved text is noisy enough to mislead [5], [6], and when the data keeps changing, as deadlines and issue states do [7].

In agentic RAG the model makes these calls itself. It decides whether to break the question up, when and how to search, whether it has enough evidence, and what is worth remembering [8]. Early methods relied on prompting [3], [9],

[10]. Newer ones learn search behaviour with reinforcement learning (RL) [11], [12].

We build the review around GRASP (GRanularity-Aware Search Policy) [13]. Most systems fix the retrieval signal and the size of the text unit before any question arrives. GRASP learns both. Its searches return single sentences, and the agent reads the surrounding paragraph only when it asks to.

Our second subject is Cortex, an assistant we built over a small team's apps. It is not an implementation of GRASP. It plans once, searches in parallel and treats each source item as a fixed unit of evidence. Because we built it on free-tier models, rate limits shaped most of its design, and setting it beside GRASP helps show which research ideas survive in that kind of setting.

Contributions. Section III sets out an eight-dimension taxonomy and reads GRASP closely. Section IV compares representative methods, Section V gives the design of Cortex with early measurements, and Section VI lists open problems.

II. BACKGROUND

A. Retrievers and Retrieval Units

BM25 ranks documents by term overlap and favours rare terms [14], so it handles exact names and identifiers well. Dense retrievers such as DPR embed questions and passages in one vector space [15] and can catch paraphrases. Static pipelines often run both and re-rank [2].

The indexed unit matters too. In Dense X, indexing short single-fact propositions instead of passages improved retrieval and QA at a fixed reader budget [16]. RAPTOR builds a tree of recursive cluster summaries for broad questions [17], and GraphRAG summarises communities in an entity graph [18]. In each case the unit is fixed at index time.

B. Limits of Single-Pass RAG

More context is not always better. Irrelevant sentences lower accuracy [5], and models underuse information placed in the

middle of long inputs [6]. Multi-hop benchmarks such as HotpotQA [19], 2WikiMultiHopQA [20] and MuSiQue [21] hide the second-hop entity until the first hop is solved, and Press et al. name the resulting failure the “compositionality gap” [4].

III. A TAXONOMY OF AGENTIC RAG

Fig. 1 groups the work into eight dimensions. A single system usually makes choices along several of them at once.

A. Query Planning and Decomposition

Self-Ask writes out follow-up questions and sends each to a search engine [4]. In Rewrite-Retrieve-Read a small rewriter is trained with RL, with the reader’s output as feedback [22], while ReAct chooses every action only after seeing the previous observation [9]. A plan made up front is predictable and parallelises easily. A step-by-step plan adapts to results but costs extra calls.

B. Iterative and Adaptive Retrieval

IRCoT uses each chain-of-thought sentence as the next query [3]. FLARE retrieves when a drafted sentence contains low-confidence tokens [10], and Iter-RetGen queries with a whole draft answer [23]. Adaptive-RAG trains a classifier that picks no, single-step or multi-step retrieval by question complexity [24]. These methods vary when retrieval happens; the retriever and chunk size stay put.

C. Search Granularity and Search Policies

Search-R1 trains an LLM with RL to search in the middle of its reasoning, rewarding only the final answer [11], and Searcho1 adds search to large reasoning models [39]. MARAG-R1 trains an agent with semantic, keyword, filter and aggregate tools [12]. A-RAG exposes keyword search, semantic search and chunk reading as separate interfaces [25]. GRASP needs a closer look.

1) Problem: Gandhi et al. observe that RL-trained retrieval agents are normally bound to one retriever and to fixed chunks of fair size [13]. The kind and amount of evidence then cannot change between steps. GRASP folds three decisions into one learned policy: how to mix lexical with dense search, how much to read, and when to stop.

2) Formulation and actions: GRASP casts agentic RAG as a finite-horizon Markov decision process[1]. Its state holds the question, the history, the evidence and tools used so far, and the remaining turn budget. Each paragraph is split into sentences that point back to it. At each step the policy reasons and then takes one action: semantic search (Qwen3-0.6B embeddings), keyword search (BM25), read paragraph (the parent of a retrieved sentence) or answer. Searches return the top K = 5 sentences, so the agent spends context on a full paragraph only by choice.

TABLE I RESULTS REPORTED BY GRASP [13]: RETRIEVAL RECALL (R) AND ANSWER F1 ON 500 VALIDATION QUESTIONS PER DATASET. COMPARE THE F1 COLUMNS: GRASP IS BEST ON ALL THREE, WHILE IRCOT LEADS

ON HOTPOTQA RECALL. NUMBERS ARE COPIED FROM THE GRASP

PAPER, NOT RE-RUN.

HotpotQA 2Wiki MuSiQue

Method R F1 R F1 R F1

Single-step hybrid 0.86 0.48 0.60 0.32 0.59 0.20 IRCoT 0.91 0.36 0.83 0.40 0.62 0.15 Search-R1 (PPO) 0.76 0.47 0.74 0.48 0.55 0.26 Search-R1 (GRPO) 0.74 0.56 0.77 0.53 0.62 0.30 Base (no RL) 0.36 0.24 0.27 0.20 0.24 0.10 GRASP 0.90 0.66 0.90 0.60 0.70 0.33

3) Reward: The trajectory reward is

$R = RA+\alpha RR+\beta RS+\gamma RE$, $(\alpha, \beta, \gamma) = (0.7, 0.15, 0.15)$.

(1) RA is answer token F1. RR is the F1 between paragraphs read and gold supporting paragraphs, which penalises reading distractors. RS is 1 only if semantic and keyword search each find a gold paragraph. RE = (T −Tcur)/T, where T is the turn budget, rewards finishing early. It is paid only when RA > 0.5, so a fast wrong answer earns nothing. The auxiliary weights sum to 1, which keeps answer F1 dominant and limits reward hacking.

4) Training and evaluation: Qwen2.5-3B-Instruct is finetuned with GRPO [40] on the HotpotQA distractor split [19] for 270 steps, only about 19% of an epoch. Episodes stop after 10 turns, and retrieved tokens are masked out of the loss. Testing uses 500 validation questions from each of HotpotQA, 2WikiMultiHopQA and MuSiQue [19]–[21]. The last two are never seen in training. Baselines are single-step lexical, semantic and hybrid retrieval, IRCoT driven by a proprietary model, the public Search-R1 PPO and GRPO checkpoints, and an untrained "Base" agent given GRASP's prompt and tools.

5) Findings: On all three datasets GRASP has the best F1, and it has the best recall on 2WikiMultiHopQA and MuSiQue (Table I). The authors put IRCoT's slightly higher HotpotQA recall down to the strong proprietary model writing its reasoning. Even single-step retrieval beats the untrained Base agent, so tools do little for a model that has not learned to use them. In a HotpotQA ablation, F1 falls only from 0.631 to 0.621 without keyword search and to 0.567 without semantic search, but removing the read action (searches then return whole paragraphs) brings it down to 0.484. The authors' reading is that a paragraph puts many entities in view at once, and the agent's next queries drift back toward the original question instead of following one clue.

6) Learned behaviour: A typical trajectory starts with semantic search. The agent reads one paragraph to pick up a bridge entity and runs a keyword search on it; if that fails, it widens to a broader semantic query. The authors liken this to skimming a page and then scanning it for a name.

7) Limitations stated by the authors: RR and RS need gold supporting facts, which few corpora have. The policy

Agentic RAG design dimensions

A. Query planning and decomposition

Self-Ask [4], query rewriting [22], ReAct [9]

B. Iterative and adaptive retrieval IRCoT [3], FLARE [10],

Iter-RetGen [23], Adaptive-RAG [24]

C. Granularity and search policy GRASP [13], SearchR1 [11], MARAG-R1 [12], A-RAG [25], Dense X [16]

D. Self-reflection and

corrective retrieval

Self-RAG [26], CRAG [27], Reflexion [28]

E. Tool use and connectors

Toolformer [29], ReAct [9], MCP [30], [31]

F. Long-term memory MemGPT [32], Generative

Agents [33], Mem0 [34]

G. Attribution and citation verification ALCE [35], AIS [36],

FActScore [37]

H. Evaluation multi-hop QA [19]–[21], RAGAs [38], FreshQA [7]

Fig. 1. The eight design dimensions used in this review, with representative (not exhaustive) methods for each. Branch C, shaded, holds the anchor work GRASP [13]. Some methods, such as ReAct, appear in two branches.

also sometimes answers too early, tool use plateaus near eight calls, and only one 3B backbone was tried. For future work they propose larger tool sets, rewards built from weaker or model-based grounding signals, and granularity that ranges from spans to whole document regions. Their ethics statement recommends provenance, citations, audit logs and human approval for consequential actions, all of which return in Section V.

D. Self-Reflection and Corrective Retrieval

Self-RAG learns reflection tokens that decide when to retrieve and grade relevance and support [26]. CRAG scores what was retrieved and switches to web search if the documents look poor [27], and Reflexion writes notes on past failures for the next attempt [28]. GRASP has no critic at inference time. The grounded-reading reward does that job, but only while the policy is trained.

E. Tool Use and Connector Protocols

Toolformer learns from a few examples where API calls help [29], and ReAct made interleaved reasoning and tool calls common [9]. Anthropic's Model Context Protocol (MCP), released in November 2024, standardises how clients discover and call external tools [30], but every server adds attack surface [31]. Indirect prompt injection becomes a concrete risk once retrieved text can trigger tool calls [41].

F. Long-Term Memory

MemGPT swaps data between the context window and external storage, much like OS paging [32]. Generative Agents score memories on recency, importance and relevance, then write reflections back [33]. Mem0 pulls facts out of conversations and consolidates them, optionally as a graph [34]. Once the data is personal, provenance, expiry and deletion cannot be afterthoughts.

G. Attribution and Citation Verification

ALCE tests whether cited passages support an LLM's statements and finds many that do not [35]. AIS defines attribution to identified sources for human raters [36], and FActScore scores atomic facts against a knowledge source [37]. Like Cortex, all three judge individual claims.

H. Evaluation

Most agentic RAG work, GRASP included, evaluates on Wikipedia multi-hop QA [19]–[21]. RAGAs scores faithfulness without reference answers [38], and FreshQA targets answers that change over time [7]. We found no benchmark for agents that work over private, changing, multi-source data and also take actions.

IV. COMPARATIVE ANALYSIS

Table II compares representative methods. "Granularity" records whether the evidence unit is fixed or chosen by the agent, and "Live data" whether sources change at question time. A dash means the work does not address that dimension.

The learned search policies (Search-R1, MARAG-R1, GRASP) were all trained on static Wikipedia corpora with gold evidence, while the methods that query live sources use prompting. Only A-RAG and GRASP let the agent choose the evidence unit at query time, and GRASP's ablation is the most direct evidence we found that this choice changes accuracy. Memory, claim verification and actions appear in none of the learned-retrieval papers. They show up in deployed systems, where cost and side effects matter as much as benchmark scores.

V. CASE STUDY: CORTEX

Cortex answers questions such as "what changed on this project this week, and what is at risk?" over a user's connected apps. Every factual sentence cites numbered evidence, unsupported claims are struck through, conflicts between sources are listed, and requested actions wait for approval. We describe only what is implemented.

TABLE II SURVEYED METHODS AND CORTEX ALONG THE TAXONOMY DIMENSIONS, AS DESCRIBED IN EACH PUBLICATION. READ DOWN A COLUMN TO SEE

WHICH DIMENSIONS ARE RARELY COMBINED: NO METHOD WITH A LEARNED POLICY ALSO HAS MEMORY, LIVE DATA OR ACTIONS.

Method Retrieval strategy Granularity Policy Memory Verification Live data Actions

RAG [1] Single-shot dense retrieval Fixed passage Fine-tuned retriever and generator

– – – –

Self-Ask [4] Explicit follow-up subquestions

Search-engine answers

Prompting – – Web search –

IRCoT [3] Retrieve at each reasoning step Fixed paragraph Prompting – – – – FLARE [10] Retrieve when draft has low confidence

Fixed passage Prompting – Regenerate uncertain sentence

– –

AdaptiveRAG [24]

Route: none, single or multistep

Fixed Trained classifier – – – –

Self-RAG [26] On demand via reflection tokens

Fixed passage Fine-tuned – Relevance and support tokens

– –

CRAG [27] Evaluator-triggered correction Strip filtering Trained evaluator – Retrieval evaluator Web fallback – ReAct [9] Interleaved thought and action API page or lookup Prompting – – Wikipedia API Environment actions Search-R1 [11] Multi-turn search in reasoning Fixed passage RL (PPO, GRPO) – – – – MARAGR1 [12]

Semantic, keyword, filter, aggregate tools

Fixed chunk SFT, then RL – – – –

A-RAG [25] Hierarchical retrieval interfaces

Keyword to chunk Model-driven tool use

– – – –

GRASP [13] Coordinated semantic and keyword search

Sentence, paragraph on demand

RL (GRPO), 4-part reward

– Grounded-reading reward (training only)

– –

MemGPT [32] Paging between context and storage

– Function calls Tiered virtual context

– – –

Mem0 [34] Memory extraction and retrieval

– – Consolidated, graph variant

– – –

Cortex (ours) LLM plan of 2–5 subquestions, parallel per-source search, one bounded retry

Fixed per source item

Rules, prompted router and planner, hard budgets

Salience, TTL, decay, pin, edit, forget

Claim-level lexical check, LLM re-check, conflicts

Six apps, web, MCP servers

Proposed; run only after approval

A. Architecture

Cortex has two modes. Deep mode plans sub-questions and asks an LLM to re-check doubtful claims. Fast mode skips both steps and saves those calls. Fig. 2 shows one Deep-mode run. The orchestrator is a hand-written loop rather than a graph framework, so every branch is visible in the code and every run has a fixed ceiling: at most 5 sub-questions, 18 tool calls, 16 evidence items (8 in Fast mode), 90 s, and 5 LLM calls in Deep mode or 3 in Fast mode.

Check and recall. An unavailable source is reported with the reason, and the run continues without it. The memory service returns up to five memories. Pinned ones are always included, and the rest are ranked by relevance times

0.5+0.5× salience.

Route. Regular expressions handle clear cases at no model cost: greetings, “remember that...”, imperatives such as “email. . . ”, and app names that narrow the search. Other questions cost one small-model call that returns a route (general, personal, web or action), the sources and the model tier. This follows Adaptive-RAG [24], with rules and prompting in place of a trained classifier. It routed all 20 of our test questions correctly.

Plan. In Deep mode one fast-tier LLM call returns JSON with a rewritten question, an optional time window and 2–5 sub-questions, each naming 1–3 sources and keywords. Invalid JSON, Fast mode and the action route fall back to one subquestion over all in-scope sources.

Search and rank. All (source, query) pairs run in parallel

with a 12 s timeout, one retry and a 90 s cache. A sub-question that finds nothing gets one broader retry at no LLM cost. Evidence is deduplicated and ranked by keyword overlap plus a recency bonus. If nothing is found, Cortex says so and stops without calling a model.

Synthesize. The model may use only the numbered evidence and must cite every factual sentence. It then lists conflicts, gaps, proposed actions, memory candidates and follow-up questions.

Verify. A cited sentence is supported if at least 40% of its content words, and every number and date, appear in the evidence it cites. Between 20% and 40% it is partial, and below 20% it is unsupported. Hedges and advice are skipped. In Deep mode one LLM call re-checks up to eight doubtful claims. Groundedness is the mean claim score, with partial support counted as half.

Actions. Proposed actions are validated against the connector’s schema, stored with an exact preview and shown as a confirmation card. Only the approval endpoint executes them. Each approval is single-use, bound to one user and expires after 30 minutes, so model text never reaches a write API directly.

Memory. A rules gate discards questions, small talk and anything that looks like a secret. Otherwise one fast-model call proposes at most two memories, each with a kind (priority, preference, fact, person or decision), a salience and a reason shown to the user. Explicit requests are stored at salience 0.95, other candidates only at 0.5 or above. A contradiction replaces an unpinned memory, and a near-duplicate reinforces it. In a

Check and recall: source

status; top-5 memories

Route: rules, else one small-model call

Plan (LLM, JSON): 2–5 sub-questions with sources and keywords

Search: all (sub-question, source) pairs in parallel; rank

Synthesize (LLM, streamed):

cite every factual sentence

Verify: lexical claim check; LLM re-check of doubtful claims

Propose actions: validate against connector schema

Remember: rules gate, then

at most 2 extracted items

Memory service

vector search or

SQLite FTS5 provenance, TTL,

decay, pin, forget

write

Connectors: Composio (6 apps),

MCP servers, Firecrawl, sample data

LLM router: free-tier

providers, failover,

cooldowns, cache

User approval: approve, edit or decline; one use,

30 min expiry

execute once

Fig. 2. One Cortex run in Deep mode, top to bottom. Solid boxes are pipeline steps; dashed boxes are services they call. The only path to a connector's write action runs through the shaded approval box, which needs a separate request from the user. If search finds no evidence, the run stops after Search without an LLM call.

scripted 30-message test this stored all 12 expected memories and no secrets. Priorities expire after 45 days, facts after 90 and decisions after 120, while preferences and people do not expire. Salience halves every 30 days unless pinned, and users can edit or delete any memory.

Oracle Agent Memory stores the memory text and serves vector search with a 0.35 distance cut-off. A SQLite ledger holds provenance, pins, expiry and decay, caps unpinned memories at 200 per user, and provides FTS5 keyword search when Oracle is not configured.

Connectors and models. Gmail, Google Calendar, Google Drive, Notion, Slack and GitHub connect through Composio. MCP servers and Firecrawl web search are configurable, and a fictional sample workspace with seeded conflicts is used for demos and evaluation. The LLM router tries free-tier providers in a fixed order (Groq, Gemini, Cerebras, OpenRouter, then Puter). A provider that hits a rate limit is put on cooldown, responses are cached, and a streamed answer switches provider only if no token has arrived yet.

Implementation. A TypeScript monorepo shares one pipeline across the Next.js app, the tests and the evaluation scripts. The memory service runs on FastAPI.

B. Mapping Cortex to the Literature

Table III links each design choice to prior work and to GRASP.

Planning and retrieval. One up-front plan, close to SelfAsk with rewriting [4], [22], fits free-tier limits and keeps

TABLE III CORTEX DESIGN CHOICES, THE WORK EACH DRAWS ON, AND HOW GRASP RELATES. THE RIGHT COLUMN SHOWS WHERE A GRASP IDEA

COULD REPLACE A FIXED RULE IN CORTEX.

Cortex choice Related work Relation to GRASP

Up-front plan of subquestions

Self-Ask [4], rewriting [22]

GRASP decides step by step; Cortex plans once for predictable cost.

Parallel search, one retry

IRCoT [3], Adaptive-RAG [24]

Multi-turn search could replace the fixed retry when a hop depends on an earlier result. Item-level evidence

Dense X [16], ARAG [25]

Fixed unit; GRASP's ablation argues for sentence hits plus an explicit "read" action. Keyword queries to each source's own search

BM25 [14], DPR [15]

One signal per source; GRASP's complementary-search idea applies.

Hard budgets FLARE [10], Adaptive-RAG [24]

Play the role of GRASP's turn-efficiency term, but fixed rather than learned. Claim verification, conflicts

ALCE [35], AIS [36], FActScore [37], CRAG [27]

Could supply the weak grounding signal GRASP lists as future work.

"Not found" answer, list of gaps

Self-RAG [26] Addresses GRASP's stated failure mode of answering too early.

Governed memory

MemGPT [32], Gen. Agents [33], Mem0 [34]

Outside GRASP's scope.

MCP connectors, approval gate

ReAct [9], MCP [30], [31], injection [41]

Follows GRASP's ethics advice on human approval for consequential actions.

latency low. The price is that no sub-question can use another's result, which is the bridge-entity step IRCoT and GRASP rely on [3], [13].

Where GRASP applies. Four GRASP ideas carry over. Read on demand. Cortex passes each source item to the synthesizer whole or truncated, so a long email thread costs a lot of context. The read-action ablation argues for returning the best-matching sentences with a pointer to the parent item and fetching the full thread only when a snippet looks relevant but incomplete. Complementary signals. Connectors mostly rely on each app's keyword search. An embedding index over fetched items would add a semantic signal, although workspace questions full of issue numbers and names may favour keywords more than Wikipedia does. Learned budgets. Cortex's hard limits play the role of GRASP's turn-efficiency term but cannot adapt to question difficulty. Weak rewards. Cortex has no gold evidence, but its verifier labels each claim against specific evidence, which is one candidate for the model-based grounding signal GRASP proposes. A policy trained on those labels could learn to please a lexical checker without becoming more correct, so human spot checks would be needed.

Beyond GRASP's setting. GRASP works on a static Wikipedia corpus. Cortex's sources fail independently, their facts go stale, they contradict each other, and the agent can change them. So Cortex reports each outage, filters by time window, lists conflicts instead of silently picking a side, and gates execution on user approval, which also limits injected instructions [31], [41].

TABLE IV PRELIMINARY CORTEX MEASUREMENTS (12 QUESTIONS, FICTIONAL

SAMPLE WORKSPACE, FREE-TIER MODELS, ONE RUN PER VERSION). LOOK AT THE ACTION AND LATENCY ROWS FOR THE EFFECT OF THE ROUTER; THE GROUNDEDNESS VALUES USE DIFFERENT CHECKERS AND SHOULD NOT BE COMPARED. THIS IS NOT A BENCHMARK COMPARISON.

Metric First + Router

Mean groundedness (lexical + LLM re-check) 0.88 0.75 Mean citation recall 0.92 0.92 Honest "not found" answers 1/1 1/1 Cross-source conflicts detected 1/2 1/2 Expected actions proposed 1/2 2/2 Actions executed without consent 0 0 Median latency 3.7 s 4.6 s 90th-percentile latency 7.1 s 6.3 s

C. Preliminary Measurements

Table IV reports two Deep-mode runs of Cortex's 12-question evaluation over its sample workspace: the first pipeline, and a later version with the router and a stricter checker. Both used free-tier models (Groq openai/gpt-oss-120b, with Gemini 3.5 Flash-Lite as fallback). Citation recall is the share of expected key facts present in the cited evidence.

These numbers are limited. We wrote the questions, the workspace is synthetic, and each version ran once. Groundedness comes from Cortex's own, partly lexical verifier, and the second run used a stricter checker, so the drop from 0.88 to 0.75 does not by itself mean answers got worse. In the first run rate limits sent 8 of 12 questions to Gemini, so model choice is confounded. No other system ran on the same data. Still, citation and verification worked end to end, Cortex said "not found" instead of guessing, no action ran without approval, and a separate action route raised correct proposals from 1/2 to 2/2. Conflict detection found only one of the two seeded conflicts, because it depends on the synthesizer noticing the disagreement. We have not built a dedicated conflict component yet.

VI. OPEN CHALLENGES AND FUTURE DIRECTIONS

Supervision without gold evidence. RL search policies need gold supporting facts [12], [13]. Verifier labels, agreement between trajectories, or AIS- and FActScore-style scores [36], [37] could substitute, with human checks against gaming.

Granularity over heterogeneous data. What the "sentence" and its "parent" should be in an email thread, a pull request or a calendar event is still open. Hierarchical units [17], [25] and propositions [16] give two places to start.

Freshness and conflict. Time-sensitive questions remain hard [7], and few methods ever show a conflict to the user. Most treat it as noise to filter out.

Verification quality. Claim checkers should be calibrated against human attribution judgements [35], [36] and should report uncertainty.

Safety and memory. Approval gates reduce injection risk without removing it [31], [41], and long-term memory [32]–

[34] raises questions of consent and deletion that no benchmark measures.

Cost-aware evaluation. GRASP settles near eight tool calls [13], but a deployed system must also budget LLM calls, much as Adaptive-RAG budgets retrieval depth [24]. Benchmarks over private, changing data should score citation support, abstention, conflicts and action safety, not only accuracy.

VII. CONCLUSION

GRASP shows that how much an agent reads, and which retrieval signal it uses, can be learned. Its 3B model beat prompted and RL-trained baselines on answer F1. Cortex instead runs over live, heterogeneous sources under tight budgets, with a fixed retrieval plan and its effort spent on verification, conflict reporting, governed memory and approvalgated actions. Combining the two needs rewards without gold evidence, evidence units for mail, chat and code, and evaluations of trust and safety.

Self-overlap with the team's own report

42 words (0.9%) of this document also occur in 8-word sequences of the report. Both documents describe the same project and were written by the same team, so this is expected. If both are submitted to the same checker, the second one checked may show this overlap; ask the coordinator to exclude the team's own submission or declare it. Longest shared passages:

Words	Page	Passage
9	5	*"gmail google calendar google drive notion slack and github"*
9	1	*"gmail google calendar google drive notion slack and github"*
8	4	*"run in parallel with a 12 s timeout"*
8	3	*"project this week and what is at risk"*
8	4	*"at most 5 sub questions 18 tool calls"*

Web exact-phrase searches

A "none" result means no page in the results contained the phrase. Topical results (pages about the same subject, e.g. the cited paper itself) are noted but are not matches. Calibration searches below show how reliable this search is.

Sent.	Page	Phrase searched	Result
0	1	*"Retrieval-augmented generation (RAG) grounds a large language model (LLM) in"*	none
5	1	*"open-source assistant we built over Gmail, Google Calendar, Google Drive"*	none
9	1	*"Index Terms—Retrieval-augmented generation, agentic RAG, large language models, search policy"*	none
17	1	*"It decides whether to break the question up, when and"*	none
24	1	*"searches return single sentences, and the agent reads the surrounding"*	none
30	1	*"Section III sets out an eight-dimension taxonomy and reads GRASP"*	none
40	1	*"Dense X, indexing short single-fact propositions instead of passages improved"*	none (topical: arXiv 2312.06648 Dense X, cited)
48	2	*"Press et al. name the resulting failure the "compositionality gap"*	none (topical: arXiv 2210.03350 Press et al., cited)
57	2	*"A plan made up front is predictable and parallelises easily"*	none
63	2	*"picks no, single-step or multi-step retrieval by question complexity"*	none (topical: Adaptive-RAG 2403.14403, cited)
69	2	*"A-RAG exposes keyword search, semantic search and chunk reading as"*	none (topical: A-RAG 2602.03442, cited)
74	2	*"actions: GRASP casts agentic RAG as a finite-horizon Markov decision"*	none (topical: GRASP 2607.10463, cited)
78	2	*"Searches return the top K = sentences, so the agent"*	not searched
88	2	*"R = RA+αRR+βRS+γRE, (α, β, γ) ="*	skipped:non-prose
93	2	*"when RA > so a fast wrong answer earns nothing"*	not searched
97	2	*"Testing uses validation questions from each of HotpotQA, and MuSiQue"*	none (topical: GRASP 2607.10463)
102	2	*"Even single-step retrieval beats the untrained Base agent, so tools"*	none (topical: GRASP 2607.10463)

106	2	*"agent reads one paragraph to pick up a bridge entity"*	none
135	3	*"The eight design dimensions used in this review, with representative"*	none
143	3	*"Self-RAG learns reflection tokens that decide when to retrieve and"*	none (topical: Self-RAG 2310.11511, cited)
150	3	*"call external tools , but every server adds attack surface"*	none
156	3	*"pulls facts out of conversations and consolidates them, optionally as"*	none
165	3	*"agentic RAG work, GRASP included, evaluates on Wikipedia multi-hop QA"*	none
172	3	*"A dash means the work does not address that dimension"*	none
176	3	*"show up in deployed systems, where cost and side effects"*	none
184	4	*"DIMENSIONS ARE RARELY COMBINED: NO METHOD WITH A LEARNED POLICY"*	skipped:table-caption
200	4	*"CRAG Evaluator-triggered correction Strip filtering Trained evaluator – Retrieval evaluator"*	skipped:non-prose
232	4	*"unavailable source is reported with the reason, and the run"*	none
238	4	*"This follows Adaptive-RAG , with rules and prompting in place"*	none
246	4	*"finds nothing gets one broader retry at no LLM cost"*	none
251	4	*"lists conflicts, gaps, proposed actions, memory candidates and follow-up questions"*	none
257	4	*"Groundedness is the mean claim score, with partial support counted"*	none
264	4	*"a kind (priority, preference, fact, person or decision), a salience"*	none
295	5	*"a connector's write action runs through the shaded approval box"*	none
299	5	*"Salience halves every days unless pinned, and users can edit"*	none
304	5	*"Firecrawl web search are configurable, and a fictional sample workspace"*	none
312	5	*"Table III links each design choice to prior work and"*	none
324	5	*"Multi-turn search could replace the fixed retry when a hop"*	none
337	5	*"Self-RAG Addresses GRASP's stated failure mode of answering too early"*	none
351	5	*"The read-action ablation argues for returning the best-matching sentences with"*	none
359	5	*"lexical checker without becoming more correct, so human spot checks"*	none
369	6	*"Mean groundedness (lexical + LLM re-check) Mean citation recall Honest"*	skipped:non-prose
376	6	*"questions, the workspace is synthetic, and each version ran once"*	none
381	6	*"two seeded conflicts, because it depends on the synthesizer noticing"*	none
392	6	*"Time-sensitive questions remain hard , and few methods ever show"*	none
400	6	*"budget LLM calls, much as Adaptive-RAG budgets retrieval depth"*	none
406	6	*"Cortex instead runs over live, heterogeneous sources under tight budgets"*	none

Calibration and targeted probes

Probe	Result
control1	found:https://arxiv.org/pdf/2005.11401:we introduce RAG models where the parametric memory is a pre-trained seq2seq model
control2	missed:Lewis et al. 2020 abstract opening 'pre-trained language models have been shown to store factual knowledge in their parameters' (true source arXiv 2005.11401 not returned)

control3	missed:Pressman risk questionnaire 'Have top software and customer managers formally committed to support the project' (textbook text; not returned)
control4	missed:SPPU template 'To develop the problem under consideration and justify feasibilty using concepts of knowledge canvas'
probe1607	none (SPPU syllabus heading 'justify feasibilty using concepts of knowledge canvas and IDEA Matrix')
probe1617	none (SPPU syllabus heading 'satisfy ability issues using modern algebra')
probe1782	none (SPPU syllabus heading 'generate various testing result charts')
probe1758	none (SPPU template 'conferences/workshops in IITs, Central Universities or UoP')

Appendix: sentence check set

Every sentence of 10 or more words (186 sentences); every 4th of them (47) was also searched on the web. Sentence numbers refer to the order of sentences in the extracted text (raw list in paper_sentences.json). Every sentence in the document, not only these, went through the source check.

Sent.	Page	Section	Sentence	Checks
0	1	Abstract	*Retrieval-augmented generation (RAG) grounds a large language model (LLM) in retrieved text, but a single retrieve-then-read pass struggles with multi-hop questions, nois…*	web + source
2	1	Abstract	*It can split a question, search more than once, pick a tool, judge its evidence and keep memory.*	source
3	1	Abstract	*We review this work around GRASP, which uses reinforcement learning to train a 3Bparameter policy that chooses, at each step, between semantic search, keyword search and …*	source
4	1	Abstract	*We sort the literature into eight design dimensions and compare representative methods along them.*	source
5	1	Abstract	*As a case study we describe Cortex, an open-source assistant we built over Gmail, Google Calendar, Google Drive, Notion, Slack and GitHub.*	web + source
6	1	Abstract	*It plans sub-questions in one LLM call, searches in parallel, checks each cited claim against its evidence, keeps memories with provenance and expiry, and runs actions on…*	source
7	1	Abstract	*On 12 questions over a synthetic workspace, the cited evidence held 92% of the expected key facts.*	source
8	1	Abstract	*Cortex declined to guess when no evidence existed, and no action ran without consent.*	source
9	1	Abstract	*Index Terms—Retrieval-augmented generation, agentic RAG, large language models, search policy, retrieval granularity, reinforcement learning, citation verification, agent…*	web + source
12	1	I. Introduction	*An LLM's knowledge is frozen at training time and goes stale.*	source
13	1	I. Introduction	*RAG conditions the answer on passages fetched from an external corpus [1], usually in one pass, with a retriever ranking chunks and a generator reading the top few [2].*	source
15	1	I. Introduction	*It fails when the second half of a question can only be searched after the first is answered [3], [4], when retrieved text is noisy enough to mislead [5], [6], and when t…*	source
17	1	I. Introduction	*It decides whether to break the question up, when and how to search, whether it has enough evidence, and what is worth remembering [8].*	web + source
20	1	I. Introduction	*Newer ones learn search behaviour with reinforcement learning (RL) [11], [12].*	source
21	1	I. Introduction	*We build the review around GRASP (GRanularity-Aware Search Policy) [13].*	source
22	1	I. Introduction	*Most systems fix the retrieval signal and the size of the text unit before any question arrives.*	source
24	1	I. Introduction	*Its searches return single sentences, and the agent reads the surrounding paragraph only when it asks to.*	web + source
25	1	I. Introduction	*Our second subject is Cortex, an assistant we built over a small team's apps.*	source
27	1	I. Introduction	*It plans once, searches in parallel and treats each source item as a fixed unit of evidence.*	source

28	1	I. Introduction	*Because we built it on free-tier models, rate limits shaped most of its design, and setting it beside GRASP helps show which research ideas survive in that kind of settin…*	source
30	1	I. Introduction	*Section III sets out an eight-dimension taxonomy and reads GRASP closely.*	web + source
31	1	I. Introduction	*Section IV compares representative methods, Section V gives the design of Cortex with early measurements, and Section VI lists open problems.*	source
36	1	II. Background	*BM25 ranks documents by term overlap and favours rare terms [14], so it handles exact names and identifiers well.*	source
37	1	II. Background	*Dense retrievers such as DPR embed questions and passages in one vector space [15] and can catch paraphrases.*	source
40	1	II. Background	*In Dense X, indexing short single-fact propositions instead of passages improved retrieval and QA at a fixed reader budget [16].*	web + source
41	1	II. Background	*RAPTOR builds a tree of recursive cluster summaries for broad questions [17], and GraphRAG summarises communities in an entity graph [18].*	source
42	1	II. Background	*In each case the unit is fixed at index time.*	source
46	1	II. Background	*Irrelevant sentences lower accuracy [5], and models underuse information placed in the*	source
48	2	II. Background	*Multi-hop benchmarks such as HotpotQA [19], 2WikiMultiHopQA [20] and MuSiQue [21] hide the second-hop entity until the first hop is solved, and Press et al. name the resu…*	web + source
52	2	III. A Taxonomy Of Agentic R	*A single system usually makes choices along several of them at once.*	source
55	2	III. A Taxonomy Of Agentic R	*Self-Ask writes out follow-up questions and sends each to a search engine [4].*	source
56	2	III. A Taxonomy Of Agentic R	*In Rewrite-Retrieve-Read a small rewriter is trained with RL, with the reader's output as feedback [22], while ReAct chooses every action only after seeing the previous o…*	source
57	2	III. A Taxonomy Of Agentic R	*A plan made up front is predictable and parallelises easily.*	web + source
58	2	III. A Taxonomy Of Agentic R	*A step-by-step plan adapts to results but costs extra calls.*	source
61	2	III. A Taxonomy Of Agentic R	*IRCoT uses each chain-of-thought sentence as the next query [3].*	source
62	2	III. A Taxonomy Of Agentic R	*FLARE retrieves when a drafted sentence contains low-confidence tokens [10], and Iter-RetGen queries with a whole draft answer [23].*	source
63	2	III. A Taxonomy Of Agentic R	*Adaptive-RAG trains a classifier that picks no, single-step or multi-step retrieval by question complexity [24].*	web + source
64	2	III. A Taxonomy Of Agentic R	*These methods vary when retrieval happens; the retriever and chunk size stay put.*	source
67	2	III. A Taxonomy Of Agentic R	*Search-R1 trains an LLM with RL to search in the middle of its reasoning, rewarding only the final answer [11], and Searcho1 adds search to large reasoning models [39].*	source
68	2	III. A Taxonomy Of Agentic R	*MARAG-R1 trains an agent with semantic, keyword, filter and aggregate tools [12].*	source
69	2	III. A Taxonomy Of Agentic R	*A-RAG exposes keyword search, semantic search and chunk reading as separate interfaces [25].*	web + source
71	2	III. A Taxonomy Of Agentic R	*1) Problem: Gandhi et al. observe that RL-trained retrieval agents are normally bound to one retriever and to fixed chunks of fair size [13].*	source

72	2	III. A Taxonomy Of Agentic R	*The kind and amount of evidence then cannot change between steps.*	source
73	2	III. A Taxonomy Of Agentic R	*GRASP folds three decisions into one learned policy: how to mix lexical with dense search, how much to read, and when to stop.*	source
74	2	III. A Taxonomy Of Agentic R	*2) Formulation and actions: GRASP casts agentic RAG as a finite-horizon Markov decision process.*	web + source
75	2	III. A Taxonomy Of Agentic R	*Its state holds the question, the history, the evidence and tools used so far, and the remaining turn budget.*	source
76	2	III. A Taxonomy Of Agentic R	*Each paragraph is split into sentences that point back to it.*	source
77	2	III. A Taxonomy Of Agentic R	*At each step the policy reasons and then takes one action: semantic search (Qwen3-0.6B embeddings), keyword search (BM25), read paragraph (the parent of a retrieved sente…*	source
78	2	III. A Taxonomy Of Agentic R	*Searches return the top K = 5 sentences, so the agent spends context on a full paragraph only by choice.*	web + source
79	2	III. A Taxonomy Of Agentic R	*TABLE I RESULTS REPORTED BY GRASP [13]: RETRIEVAL RECALL (R) AND ANSWER F1 ON 500 VALIDATION QUESTIONS PER DATASET.*	source
80	2	III. A Taxonomy Of Agentic R	*COMPARE THE F1 COLUMNS: GRASP IS BEST ON ALL THREE, WHILE IRCOT LEADS*	source
86	2	III. A Taxonomy Of Agentic R	*Single-step hybrid 0.86 0.48 0.60 0.32 0.59 0.20 IRCoT 0.91 0.36 0.83 0.40 0.62 0.15 Search-R1 (PPO) 0.76 0.47 0.74 0.48 0.55 0.26 Search-R1 (GRPO) 0.74 0.56 0.77 0.53 0.…*	source
88	2	III. A Taxonomy Of Agentic R	*$R = RA+\alpha RR+\beta RS+\gamma RE$, $(\alpha, \beta, \gamma) = (0.7, 0.15, 0.15)$.*	web + source
90	2	III. A Taxonomy Of Agentic R	*RR is the F1 between paragraphs read and gold supporting paragraphs, which penalises reading distractors.*	source
91	2	III. A Taxonomy Of Agentic R	*RS is 1 only if semantic and keyword search each find a gold paragraph.*	source
92	2	III. A Taxonomy Of Agentic R	*$RE = (T - Tcur)/T$, where T is the turn budget, rewards finishing early.*	source
93	2	III. A Taxonomy Of Agentic R	*It is paid only when $RA > 0.5$, so a fast wrong answer earns nothing.*	web + source
94	2	III. A Taxonomy Of Agentic R	*The auxiliary weights sum to 1, which keeps answer F1 dominant and limits reward hacking.*	source
95	2	III. A Taxonomy Of Agentic R	*4) Training and evaluation: Qwen2.5-3B-Instruct is finetuned with GRPO [40] on the HotpotQA distractor split [19] for 270 steps, only about 19% of an epoch.*	source
96	2	III. A Taxonomy Of Agentic R	*Episodes stop after 10 turns, and retrieved tokens are masked out of the loss.*	source
97	2	III. A Taxonomy Of Agentic R	*Testing uses 500 validation questions from each of HotpotQA, 2WikiMultiHopQA and MuSiQue [19]–[21].*	web + source
99	2	III. A Taxonomy Of Agentic R	*Baselines are single-step lexical, semantic and hybrid retrieval, IRCoT driven by a proprietary model, the public Search-R1 PPO and GRPO checkpoints, and an untrained "Ba…*	source
100	2	III. A Taxonomy Of Agentic R	*5) Findings: On all three datasets GRASP has the best F1, and it has the best recall on 2WikiMultiHopQA and MuSiQue (Table I).*	source
101	2	III. A Taxonomy Of Agentic R	*The authors put IRCoT's slightly higher HotpotQA recall down to the strong proprietary model writing its reasoning.*	source

102	2	III. A Taxonomy Of Agentic R	*Even single-step retrieval beats the untrained Base agent, so tools do little for a model that has not learned to use them.*	web + source
103	2	III. A Taxonomy Of Agentic R	*In a HotpotQA ablation, F1 falls only from 0.631 to 0.621 without keyword search and to 0.567 without semantic search, but removing the read action (searches then return …*	source
104	2	III. A Taxonomy Of Agentic R	*The authors' reading is that a paragraph puts many entities in view at once, and the agent's next queries drift back toward the original question instead of following one…*	source
105	2	III. A Taxonomy Of Agentic R	*6) Learned behaviour: A typical trajectory starts with semantic search.*	source
106	2	III. A Taxonomy Of Agentic R	*The agent reads one paragraph to pick up a bridge entity and runs a keyword search on it; if that fails, it widens to a broader semantic query.*	web + source
107	2	III. A Taxonomy Of Agentic R	*The authors liken this to skimming a page and then scanning it for a name.*	source
108	2	III. A Taxonomy Of Agentic R	*7) Limitations stated by the authors: RR and RS need gold supporting facts, which few corpora have.*	source
118	3	III. A Taxonomy Of Agentic R	*Granularity and search policy GRASP [13], SearchR1 [11], MARAG-R1 [12], A-RAG [25], Dense X [16]*	source
135	3	III. A Taxonomy Of Agentic R	*The eight design dimensions used in this review, with representative (not exhaustive) methods for each.*	web + source
138	3	III. A Taxonomy Of Agentic R	*also sometimes answers too early, tool use plateaus near eight calls, and only one 3B backbone was tried.*	source
139	3	III. A Taxonomy Of Agentic R	*For future work they propose larger tool sets, rewards built from weaker or model-based grounding signals, and granularity that ranges from spans to whole document region…*	source
140	3	III. A Taxonomy Of Agentic R	*Their ethics statement recommends provenance, citations, audit logs and human approval for consequential actions, all of which return in Section V.*	source
143	3	III. A Taxonomy Of Agentic R	*Self-RAG learns reflection tokens that decide when to retrieve and grade relevance and support [26].*	web + source
144	3	III. A Taxonomy Of Agentic R	*CRAG scores what was retrieved and switches to web search if the documents look poor [27], and Reflexion writes notes on past failures for the next attempt [28].*	source
146	3	III. A Taxonomy Of Agentic R	*The grounded-reading reward does that job, but only while the policy is trained.*	source
149	3	III. A Taxonomy Of Agentic R	*Toolformer learns from a few examples where API calls help [29], and ReAct made interleaved reasoning and tool calls common [9].*	source
150	3	III. A Taxonomy Of Agentic R	*Anthropic's Model Context Protocol (MCP), released in November 2024, standardises how clients discover and call external tools [30], but every server adds attack surface …*	web + source
151	3	III. A Taxonomy Of Agentic R	*Indirect prompt injection becomes a concrete risk once retrieved text can trigger tool calls [41].*	source
154	3	III. A Taxonomy Of Agentic R	*MemGPT swaps data between the context window and external storage, much like OS paging [32].*	source
155	3	III. A Taxonomy Of Agentic R	*Generative Agents score memories on recency, importance and relevance, then write reflections back [33].*	source
156	3	III. A Taxonomy Of Agentic R	*Mem0 pulls facts out of conversations and consolidates them, optionally as a graph [34].*	web + source
157	3	III. A Taxonomy Of Agentic R	*Once the data is personal, provenance, expiry and deletion cannot be afterthoughts.*	source

160	3	III. A Taxonomy Of Agentic R	*ALCE tests whether cited passages support an LLM's statements and finds many that do not [35].*	source
161	3	III. A Taxonomy Of Agentic R	*AIS defines attribution to identified sources for human raters [36], and FActScore scores atomic facts against a knowledge source [37].*	source
165	3	III. A Taxonomy Of Agentic R	*Most agentic RAG work, GRASP included, evaluates on Wikipedia multi-hop QA [19]–[21].*	web + source
166	3	III. A Taxonomy Of Agentic R	*RAGAs scores faithfulness without reference answers [38], and FreshQA targets answers that change over time [7].*	source
167	3	III. A Taxonomy Of Agentic R	*We found no benchmark for agents that work over private, changing, multi-source data and also take actions.*	source
171	3	IV. Comparative Analysis	*"Granularity" records whether the evidence unit is fixed or chosen by the agent, and "Live data" whether sources change at question time.*	source
172	3	IV. Comparative Analysis	*A dash means the work does not address that dimension.*	web + source
173	3	IV. Comparative Analysis	*The learned search policies (Search-R1, MARAG-R1, GRASP) were all trained on static Wikipedia corpora with gold evidence, while the methods that query live sources use pr…*	source
174	3	IV. Comparative Analysis	*Only A-RAG and GRASP let the agent choose the evidence unit at query time, and GRASP's ablation is the most direct evidence we found that this choice changes accuracy.*	source
175	3	IV. Comparative Analysis	*Memory, claim verification and actions appear in none of the learned-retrieval papers.*	source
176	3	IV. Comparative Analysis	*They show up in deployed systems, where cost and side effects matter as much as benchmark scores.*	web + source
179	3	V. Case Study: Cortex	*Cortex answers questions such as "what changed on this project this week, and what is at risk?" over a user's connected apps.*	source
180	3	V. Case Study: Cortex	*Every factual sentence cites numbered evidence, unsupported claims are struck through, conflicts between sources are listed, and requested actions wait for approval.*	source
182	4	V. Case Study: Cortex	*TABLE II SURVEYED METHODS AND CORTEX ALONG THE TAXONOMY DIMENSIONS, AS DESCRIBED IN EACH PUBLICATION.*	source
184	4	V. Case Study: Cortex	*WHICH DIMENSIONS ARE RARELY COMBINED: NO METHOD WITH A LEARNED POLICY ALSO HAS MEMORY, LIVE DATA OR ACTIONS.*	web + source
185	4	V. Case Study: Cortex	*Method Retrieval strategy Granularity Policy Memory Verification Live data Actions*	source
186	4	V. Case Study: Cortex	*RAG [1] Single-shot dense retrieval Fixed passage Fine-tuned retriever and generator*	source
191	4	V. Case Study: Cortex	*IRCoT [3] Retrieve at each reasoning step Fixed paragraph Prompting – – – – FLARE [10] Retrieve when draft has low confidence*	source
200	4	V. Case Study: Cortex	*CRAG [27] Evaluator-triggered correction Strip filtering Trained evaluator – Retrieval evaluator Web fallback – ReAct [9] Interleaved thought and action API page or looku…*	web + source
217	4	V. Case Study: Cortex	*Cortex (ours) LLM plan of 2–5 subquestions, parallel per-source search, one bounded retry*	source
227	4	V. Case Study: Cortex	*Deep mode plans sub-questions and asks an LLM to re-check doubtful claims.*	source
230	4	V. Case Study: Cortex	*The orchestrator is a hand-written loop rather than a graph framework, so every branch is visible in the code and every run has a fixed ceiling: at most 5 sub-questions, …*	source

232	4	V. Case Study: Cortex	*An unavailable source is reported with the reason, and the run continues without it.*	web + source
234	4	V. Case Study: Cortex	*Pinned ones are always included, and the rest are ranked by relevance times 0.5+0.5× salience.*	source
236	4	V. Case Study: Cortex	*Regular expressions handle clear cases at no model cost: greetings, "remember that...", imperatives such as "email. . . ", and app names that narrow the search.*	source
237	4	V. Case Study: Cortex	*Other questions cost one small-model call that returns a route (general, personal, web or action), the sources and the model tier.*	source
238	4	V. Case Study: Cortex	*This follows Adaptive-RAG [24], with rules and prompting in place of a trained classifier.*	web + source
241	4	V. Case Study: Cortex	*In Deep mode one fast-tier LLM call returns JSON with a rewritten question, an optional time window and 2–5 sub-questions, each naming 1–3 sources and keywords.*	source
242	4	V. Case Study: Cortex	*Invalid JSON, Fast mode and the action route fall back to one subquestion over all in-scope sources.*	source
245	4	V. Case Study: Cortex	*with a 12 s timeout, one retry and a 90 s cache.*	source
246	4	V. Case Study: Cortex	*A sub-question that finds nothing gets one broader retry at no LLM cost.*	web + source
247	4	V. Case Study: Cortex	*Evidence is deduplicated and ranked by keyword overlap plus a recency bonus.*	source
248	4	V. Case Study: Cortex	*If nothing is found, Cortex says so and stops without calling a model.*	source
250	4	V. Case Study: Cortex	*The model may use only the numbered evidence and must cite every factual sentence.*	source
251	4	V. Case Study: Cortex	*It then lists conflicts, gaps, proposed actions, memory candidates and follow-up questions.*	web + source
253	4	V. Case Study: Cortex	*A cited sentence is supported if at least 40% of its content words, and every number and date, appear in the evidence it cites.*	source
254	4	V. Case Study: Cortex	*Between 20% and 40% it is partial, and below 20% it is unsupported.*	source
256	4	V. Case Study: Cortex	*In Deep mode one LLM call re-checks up to eight doubtful claims.*	source
257	4	V. Case Study: Cortex	*Groundedness is the mean claim score, with partial support counted as half.*	web + source
259	4	V. Case Study: Cortex	*Proposed actions are validated against the connector's schema, stored with an exact preview and shown as a confirmation card.*	source
261	4	V. Case Study: Cortex	*Each approval is single-use, bound to one user and expires after 30 minutes, so model text never reaches a write API directly.*	source
263	4	V. Case Study: Cortex	*A rules gate discards questions, small talk and anything that looks like a secret.*	source
264	4	V. Case Study: Cortex	*Otherwise one fast-model call proposes at most two memories, each with a kind (priority, preference, fact, person or decision), a salience and a reason shown to the user.*	web + source
265	4	V. Case Study: Cortex	*Explicit requests are stored at salience 0.95, other candidates only at 0.5 or above.*	source
266	4	V. Case Study: Cortex	*A contradiction replaces an unpinned memory, and a near-duplicate reinforces it.*	source

294	5	V. Case Study: Cortex	*Solid boxes are pipeline steps; dashed boxes are services they call.*	source
295	5	V. Case Study: Cortex	*The only path to a connector's write action runs through the shaded approval box, which needs a separate request from the user.*	web + source
296	5	V. Case Study: Cortex	*If search finds no evidence, the run stops after Search without an LLM call.*	source
297	5	V. Case Study: Cortex	*scripted 30-message test this stored all 12 expected memories and no secrets.*	source
298	5	V. Case Study: Cortex	*Priorities expire after 45 days, facts after 90 and decisions after 120, while preferences and people do not expire.*	source
299	5	V. Case Study: Cortex	*Salience halves every 30 days unless pinned, and users can edit or delete any memory.*	web + source
300	5	V. Case Study: Cortex	*Oracle Agent Memory stores the memory text and serves vector search with a 0.35 distance cut-off.*	source
301	5	V. Case Study: Cortex	*A SQLite ledger holds provenance, pins, expiry and decay, caps unpinned memories at 200 per user, and provides FTS5 keyword search when Oracle is not configured.*	source
303	5	V. Case Study: Cortex	*Gmail, Google Calendar, Google Drive, Notion, Slack and GitHub connect through Composio.*	source
304	5	V. Case Study: Cortex	*MCP servers and Firecrawl web search are configurable, and a fictional sample workspace with seeded conflicts is used for demos and evaluation.*	web + source
305	5	V. Case Study: Cortex	*The LLM router tries free-tier providers in a fixed order (Groq, Gemini, Cerebras, OpenRouter, then Puter).*	source
306	5	V. Case Study: Cortex	*A provider that hits a rate limit is put on cooldown, responses are cached, and a streamed answer switches provider only if no token has arrived yet.*	source
308	5	V. Case Study: Cortex	*A TypeScript monorepo shares one pipeline across the Next.js app, the tests and the evaluation scripts.*	source
312	5	V. Case Study: Cortex	*Table III links each design choice to prior work and to GRASP.*	web + source
314	5	V. Case Study: Cortex	*One up-front plan, close to SelfAsk with rewriting [4], [22], fits free-tier limits and keeps*	source
315	5	V. Case Study: Cortex	*TABLE III CORTEX DESIGN CHOICES, THE WORK EACH DRAWS ON, AND HOW GRASP RELATES.*	source
321	5	V. Case Study: Cortex	*GRASP decides step by step; Cortex plans once for predictable cost.*	source
324	5	V. Case Study: Cortex	*Multi-turn search could replace the fixed retry when a hop depends on an earlier result.*	web + source
327	5	V. Case Study: Cortex	*Fixed unit; GRASP's ablation argues for sentence hits plus an explicit "read" action.*	source
332	5	V. Case Study: Cortex	*Play the role of GRASP's turn-efficiency term, but fixed rather than learned.*	source
335	5	V. Case Study: Cortex	*Could supply the weak grounding signal GRASP lists as future work.*	source
337	5	V. Case Study: Cortex	*Self-RAG [26] Addresses GRASP's stated failure mode of answering too early.*	web + source
344	5	V. Case Study: Cortex	*Follows GRASP's ethics advice on human approval for consequential actions.*	source

346	5	V. Case Study: Cortex	*The price is that no sub-question can use another's result, which is the bridge-entity step IRCoT and GRASP rely on [3], [13].*	source
350	5	V. Case Study: Cortex	*Cortex passes each source item to the synthesizer whole or truncated, so a long email thread costs a lot of context.*	source
351	5	V. Case Study: Cortex	*The read-action ablation argues for returning the best-matching sentences with a pointer to the parent item and fetching the full thread only when a snippet looks relevan…*	web + source
354	5	V. Case Study: Cortex	*An embedding index over fetched items would add a semantic signal, although workspace questions full of issue numbers and names may favour keywords more than Wikipedia do…*	source
356	5	V. Case Study: Cortex	*Cortex's hard limits play the role of GRASP's turn-efficiency term but cannot adapt to question difficulty.*	source
358	5	V. Case Study: Cortex	*Cortex has no gold evidence, but its verifier labels each claim against specific evidence, which is one candidate for the model-based grounding signal GRASP proposes.*	source
359	5	V. Case Study: Cortex	*A policy trained on those labels could learn to please a lexical checker without becoming more correct, so human spot checks would be needed.*	web + source
362	5	V. Case Study: Cortex	*Cortex's sources fail independently, their facts go stale, they contradict each other, and the agent can change them.*	source
363	5	V. Case Study: Cortex	*So Cortex reports each outage, filters by time window, lists conflicts instead of silently picking a side, and gates execution on user approval, which also limits injecte…*	source
366	6	V. Case Study: Cortex	*LOOK AT THE ACTION AND LATENCY ROWS FOR THE EFFECT OF THE ROUTER; THE GROUNDEDNESS VALUES USE DIFFERENT CHECKERS AND SHOULD NOT BE COMPARED.*	source
369	6	V. Case Study: Cortex	*Mean groundedness (lexical + LLM re-check) 0.88 0.75 Mean citation recall 0.92 0.92 Honest "not found" answers 1/1 1/1 Cross-source conflicts detected 1/2 1/2 Expected ac…*	web + source
372	6	V. Case Study: Cortex	*Table IV reports two Deep-mode runs of Cortex's 12-question evaluation over its sample workspace: the first pipeline, and a later version with the router and a stricter c…*	source
373	6	V. Case Study: Cortex	*Both used free-tier models (Groq openai/gpt-oss-120b, with Gemini 3.5 Flash-Lite as fallback).*	source
374	6	V. Case Study: Cortex	*Citation recall is the share of expected key facts present in the cited evidence.*	source
376	6	V. Case Study: Cortex	*We wrote the questions, the workspace is synthetic, and each version ran once.*	web + source
377	6	V. Case Study: Cortex	*Groundedness comes from Cortex's own, partly lexical verifier, and the second run used a stricter checker, so the drop from 0.88 to 0.75 does not by itself mean answers g…*	source
378	6	V. Case Study: Cortex	*In the first run rate limits sent 8 of 12 questions to Gemini, so model choice is confounded.*	source
380	6	V. Case Study: Cortex	*Still, citation and verification worked end to end, Cortex said "not found" instead of guessing, no action ran without approval, and a separate action route raised correc…*	source
381	6	V. Case Study: Cortex	*Conflict detection found only one of the two seeded conflicts, because it depends on the synthesizer noticing the disagreement.*	web + source
387	6	VI. Open Challenges And Futu	*Verifier labels, agreement between trajectories, or AIS- and FActScore-style scores [36], [37] could substitute, with human checks against gaming.*	source

389	6	VI. Open Challenges And Futu	*What the “sentence” and its “parent” should be in an email thread, a pull request or a calendar event is still open.*	source
390	6	VI. Open Challenges And Futu	*Hierarchical units [17], [25] and propositions [16] give two places to start.*	source
392	6	VI. Open Challenges And Futu	*Time-sensitive questions remain hard [7], and few methods ever show a conflict to the user.*	web + source
395	6	VI. Open Challenges And Futu	*Claim checkers should be calibrated against human attribution judgements [35], [36] and should report uncertainty.*	source
397	6	VI. Open Challenges And Futu	*Approval gates reduce injection risk without removing it [31], [41], and long-term memory [32]–*	source
398	6	VI. Open Challenges And Futu	*[34] raises questions of consent and deletion that no benchmark measures.*	source
400	6	VI. Open Challenges And Futu	*GRASP settles near eight tool calls [13], but a deployed system must also budget LLM calls, much as Adaptive-RAG budgets retrieval depth [24].*	web + source
401	6	VI. Open Challenges And Futu	*Benchmarks over private, changing data should score citation support, abstention, conflicts and action safety, not only accuracy.*	source
404	6	VII. Conclusion	*GRASP shows that how much an agent reads, and which retrieval signal it uses, can be learned.*	source
405	6	VII. Conclusion	*Its 3B model beat prompted and RL-trained baselines on answer F1.*	source
406	6	VII. Conclusion	*Cortex instead runs over live, heterogeneous sources under tight budgets, with a fixed retrieval plan and its effort spent on verification, conflict reporting, governed m…*	web + source
407	6	VII. Conclusion	*Combining the two needs rewards without gold evidence, evidence units for mail, chat and code, and evaluations of trust and safety.*	source

Method, tools and limitations

- **Text extraction:** PyMuPDF 1.28.5 (Python bindings 2.0, Python 3.11.9) from the submitted PDF; hyphenated line breaks joined; running headers, footers and page numbers removed.
- **Matching:** lower-cased word tokens [A-Za-z0-9]+; a word counts as matched if it lies inside any 8-word sequence that also occurs in a source. Shorter shared phrases (technical terms, method names) are not counted, as in most checkers' default "exclude small matches" setting.
- **Sources:** Semantic Scholar Graph API (paper/search/match) for titles and abstracts; arXiv export mirror for full texts; fetched 31 October 2026.
- **Web:** Claude WebSearch tool, one quoted phrase per query (combined OR queries were rejected by the tool).
- **Rendering:** HTML printed to PDF with Google Chrome 156 headless.
- **Scripts and raw data** (extract.py, sentences.py, fetch_sources.py, analyze.py, build_report.py, *_sentences.json, web_log.jsonl, sources.json) are kept with the team for re-running.

Limitations

- No repository check. Turnitin and DrillBit also compare against their stores of student submissions, theses and paywalled journals. This report cannot, so the official index can be higher, mainly from SPPU template wording (lab-assignment and annexure headings) that appears in many earlier BE reports.
- The web search is not a plagiarism engine. In calibration it found the opening of the RAG paper abstract by one phrase but missed it by another, and missed a Pressman textbook question and an SPPU syllabus heading that certainly exist. A web "none" is weak evidence; the source check over the cited papers is the stronger part.
- Paywalled sources (ACL/ACM/IEEE versions, textbooks such as Pressman & Maxim, McKinsey reports) were compared only by title and abstract, or by their arXiv preprint.
- Paraphrase and translation are not detected; only identical 8-word runs are.
